Sygnały i Obrazy Cyfrowe

Ćwiczenie nr 2. Demozaikowanie

Michał Zywer, nr albumu 290061

12 listopada 2026

Spis treści

1 Wstęp **2**

2 Symulacja matrycy z filtrem CFA (mozaikowanie) **3**

3 Algorytm demozaikowania **5**
3.1 Znormalizowany splot 5
3.2 Wpływ skali s 6

4 Wyniki demozaikowania **6**

5 Analiza wyników **11**
5.1 Miary błędu 11
5.2 Bayer 13
5.3 X-Trans 13
5.4 Porównanie filtrów Bayera i X-Trans 13

6 Wnioski **14**

Spis rysunków

1 Oryginalny obraz testowy. 3
2 Obrazy RAW z matrycy z filtrem CFA Bayera 4
3 Obrazy RAW z matrycy z filtrem CFA X-Trans 5
4 Bayer. Demozaikowanie algorytmem Okno, $s = 2{,}5$ 7
5 Bayer. Demozaikowanie algorytmem Namiot, $s = 1{,}5$ 7
6 Bayer. Demozaikowanie algorytmem Keys'a, $s = 2{,}0$ 8
7 X-Trans. Demozaikowanie algorytmem Okno, $s = 2{,}5$ 8
8 X-Trans. Demozaikowanie algorytmem Namiot, $s = 1{,}5$ 9
9 X-Trans. Demozaikowanie algorytmem Keys'a, $s = 2{,}5$ 9
10 Bayer. Powiększony fragment napisu (320×180 pikseli) po demozaikowaniu 10
11 X-Trans. Powiększony fragment napisu (320×180 pikseli) po demozaikowaniu 10

12 Powiększony fragment klina rozdzielczości (240 × 135 pikseli) 11

Spis tabel

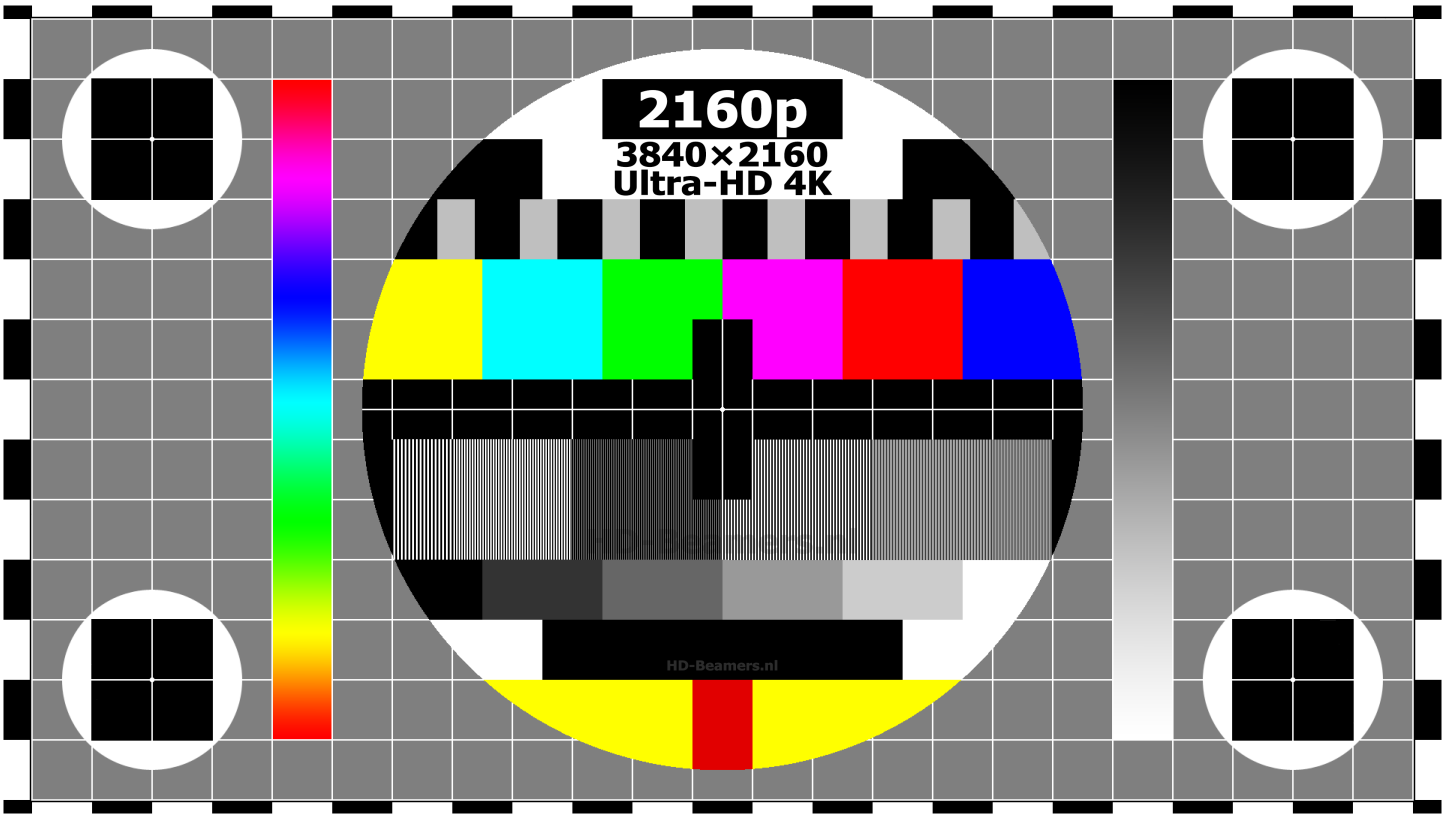
1 Bayer. Porównanie obrazów zdemozaikowanych z obrazem oryginalnym 12
2 X-Trans. Porównanie obrazów zdemozaikowanych z obrazem oryginalnym 12
3 Niskoczęstotliwościowy błąd barwy (barwna mora): RMSE różnic chrominancji uśrednionych w oknie 12 × 12 pikseli 14

1 Wstęp

Do napisania tego sprawozdania użyto narzędzi AI.

Kod źródłowy programów użytych do mozaikowania oraz demozaikowania obrazu można znaleźć na mojej stronie Github: `https://github.com/n0rmii/SIOC/tree/main/L03`. Są one napisane w języku C, a do renderowania używają biblioteki SDL2. Program demozaikujący jest zmodyfikowaną wersją programu do skalowania z poprzedniego ćwiczenia i korzysta z tych samych funkcji interpolujących: „Najbliższy sąsiad" (tzw. „Okno", Π), liniowej (tzw. „Namiot", Λ) oraz sześciennej interpolacyjnej R. Keys'a (`https://ncorr.com/download/publications/keysbicubic.pdf`).

Zadanie polegało na (i) zasymulowaniu działania matrycy CMOS z filtrem kolorów CFA (ang. *Color Filter Array*) Bayera oraz Fuji X-Trans i (ii) odtworzeniu z tak uzyskanego obrazu RAW pełnego obrazu RGB za pomocą wymienionych funkcji interpolujących. Wszystkie operacje wykonano na tym samym obrazie testowym co w poprzednim ćwiczeniu, pochodzącym ze strony HD-Beamers.nl:



Rysunek 1: Oryginalny obraz testowy.

2 Symulacja matrycy z filtrem CFA (mozaikowanie)

Każdy piksel matrycy rejestruje tylko jedną składową koloru, wyznaczoną przez filtr leżący nad nim. Mozaikowanie polega więc na wyzerowaniu w każdym pikselu dwóch składowych, których filtr nie przepuszcza. Wzór filtru jest zapisany jako trzy maski binarne $M_c(x,y) \in \{0,1\}$, $c \in \{R,G,B\}$, powtarzane okresowo co 2×2 (Bayer) lub 6×6 (X-Trans) pikseli:

$$S_c(x,y) = M_c(x \bmod w,\, y \bmod h) \cdot I_c(x,y),$$

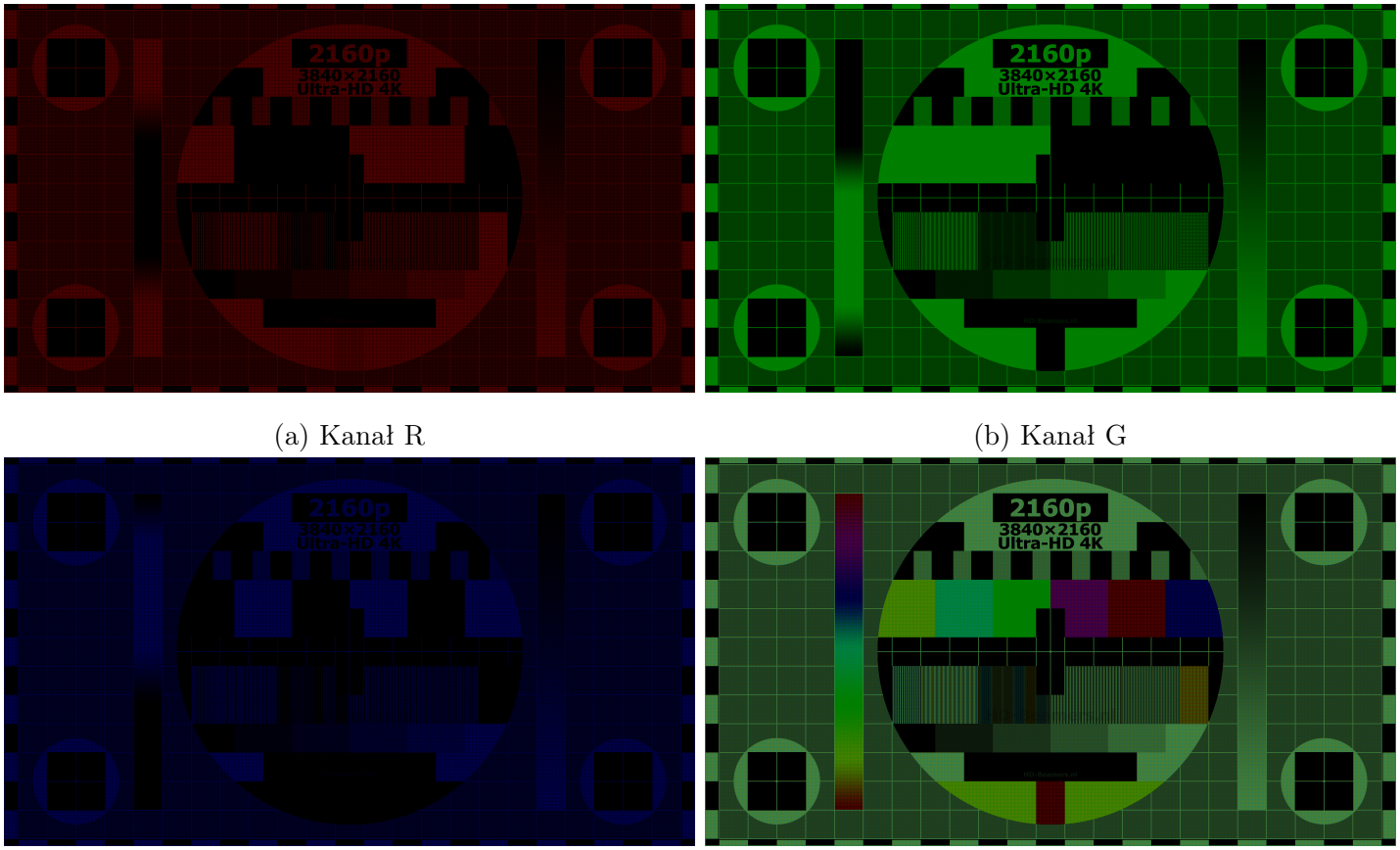
gdzie I_c to kanał obrazu oryginalnego, a $w \times h$ to wymiary wzoru.

Zastosowano następujące wzory filtrów:

$$\text{Bayer: } \begin{bmatrix} R & G \\ G & B \end{bmatrix} \qquad \text{X-Trans: } \begin{bmatrix} G & B & R & G & R & B \\ R & G & G & B & G & G \\ B & G & G & R & G & G \\ G & R & B & G & B & R \\ B & G & G & R & G & G \\ R & G & G & B & G & G \end{bmatrix}$$

Wzór X-Trans jest zgodny z instrukcją. Wzór Bayera w programie ma układ RGGB, a w instrukcji GRBG. Oba różnią się tylko przesunięciem o jeden piksel w poziomie, co dla opisanego niżej algorytmu demozaikowania nie ma znaczenia (poza samą krawędzią obrazu).

W filtrze Bayera piksele zielone stanowią 1/2, a czerwone i niebieskie po 1/4 wszystkich pikseli. Próbki R i B leżą na regularnej siatce o okresie 2 piksele, a G na siatce szachownicowej. W filtrze X-Trans zielonych jest $20/36 \approx 56\%$, a czerwonych i niebieskich po $8/36 \approx 22\%$. Ich rozmieszczenie jest nieregularne, ale w każdym wierszu i każdej kolumnie występują wszystkie trzy kolory, a w każdym oknie 3×3 jest co najmniej jeden piksel R, G i B.



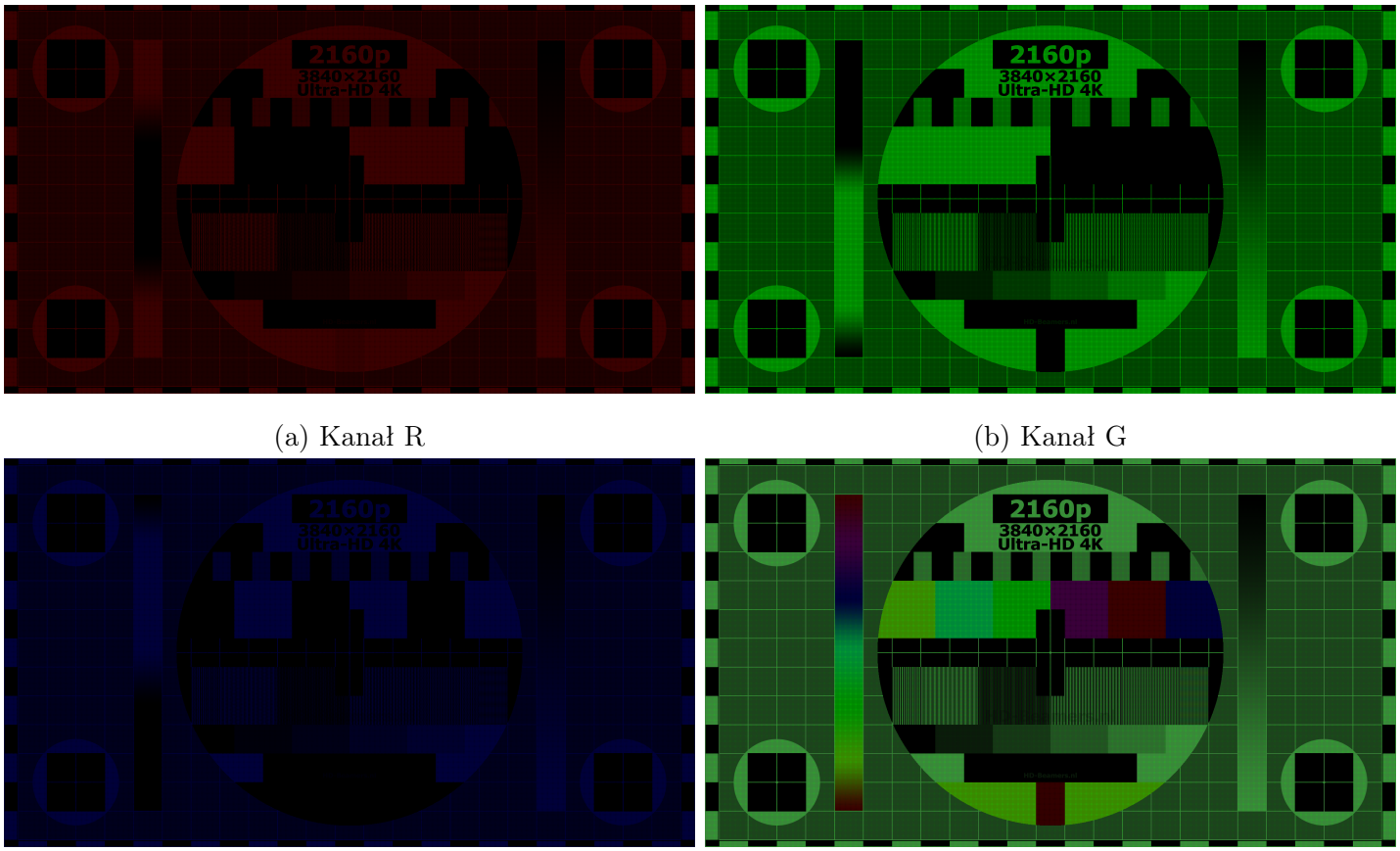
(a) Kanał R

(b) Kanał G

(c) Kanał B

(d) Obraz matrycy (RGB)

Rysunek 2: Obrazy RAW z matrycy z filtrem CFA Bayera



(a) Kanał R

(b) Kanał G

(c) Kanał B

(d) Obraz matrycy (RGB)

Rysunek 3: Obrazy RAW z matrycy z filtrem CFA X-Trans

Obrazy pojedynczych kanałów (rys. 2 i 3) są wyraźnie ciemniejsze od oryginału, bo większość ich pikseli jest wyzerowana: w kanałach R i B jest to 3/4 pikseli dla Bayera i 7/9 dla X-Trans. Na obrazie matrycy (RGB) widać charakterystyczną drobną kratkę: w przypadku Bayera regularną, a w przypadku X-Trans o bardziej złożonym, mniej regularnym wzorze.

3 Algorytm demozaikowania

3.1 Znormalizowany splot

Najprostszym podejściem byłoby zmniejszenie obrazu o wymiary wzoru CFA (każdy blok 2×2 lub 6×6 dałby jeden piksel RGB). Traci się jednak wtedy większość rozdzielczości, a celem demozaikowania jest obraz RGB w pełnej rozdzielczości matrycy. Dlatego każdy kanał uzupełniono osobno, interpolując brakujące piksele z sąsiednich próbek tego samego koloru.

Ponieważ próbki danego koloru nie leżą na pełnej siatce (a w przypadku X-Trans nawet na regularnej), zwykła interpolacja z poprzedniego ćwiczenia nie może być zastosowana wprost. Zastosowano *znormalizo-*

wany splot (ang. *normalized convolution*):

$$\hat{I}_c(x,y) = \frac{\sum_{i,j} K\left(\frac{i}{s}\right) K\left(\frac{j}{s}\right) M_c(x+i, y+j)\, I_c(x+i, y+j)}{\sum_{i,j} K\left(\frac{i}{s}\right) K\left(\frac{j}{s}\right) M_c(x+i, y+j)},$$

gdzie K to funkcja interpolująca (Π, Λ lub Keys'a), a s to jej skala (rozciągnięcie), podawana jako parametr programu. Licznik to ważona suma *istniejących* próbek koloru c w otoczeniu piksela, a mianownik to suma wag tych próbek. Iloraz jest więc średnią ważoną, która nie zależy od tego, ile próbek wypadło w oknie. Wzór CFA wchodzi do obliczeń tylko przez maskę M_c, dzięki czemu ten sam kod obsługuje oba filtry.

Funkcja interpolująca jest separowalna, więc obliczenia wykonano w dwóch przebiegach 1D, tak jak przy skalowaniu. W przebiegu poziomym dla każdego kanału liczona jest osobno ważona suma próbek i suma wag, zapisywane w tablicach zmiennoprzecinkowych (`double`). W przebiegu pionowym obie wielkości są splatane z tą samą funkcją i dopiero na końcu dzielone przez siebie. Wynik jest zaokrąglany i ograniczany do zakresu 0–255 tylko raz (`clampToByte`).

Jeśli w oknie funkcji nie ma żadnej próbki danego koloru, mianownik jest równy zeru. Taki przypadek jest wykrywany progiem: gdy suma wag jest mniejsza niż 10^{-6}, wynikowi przypisywana jest wartość 0. Przy poprawnie dobranej skali s sytuacja ta nie występuje.

3.2 Wpływ skali s

Program liczy wagi tylko w całkowitych odległościach t od piksela, czyli $K(t/s)$. Dla funkcji Okno (przedział $[-0{,}5;\ 0{,}5)$) daje to:

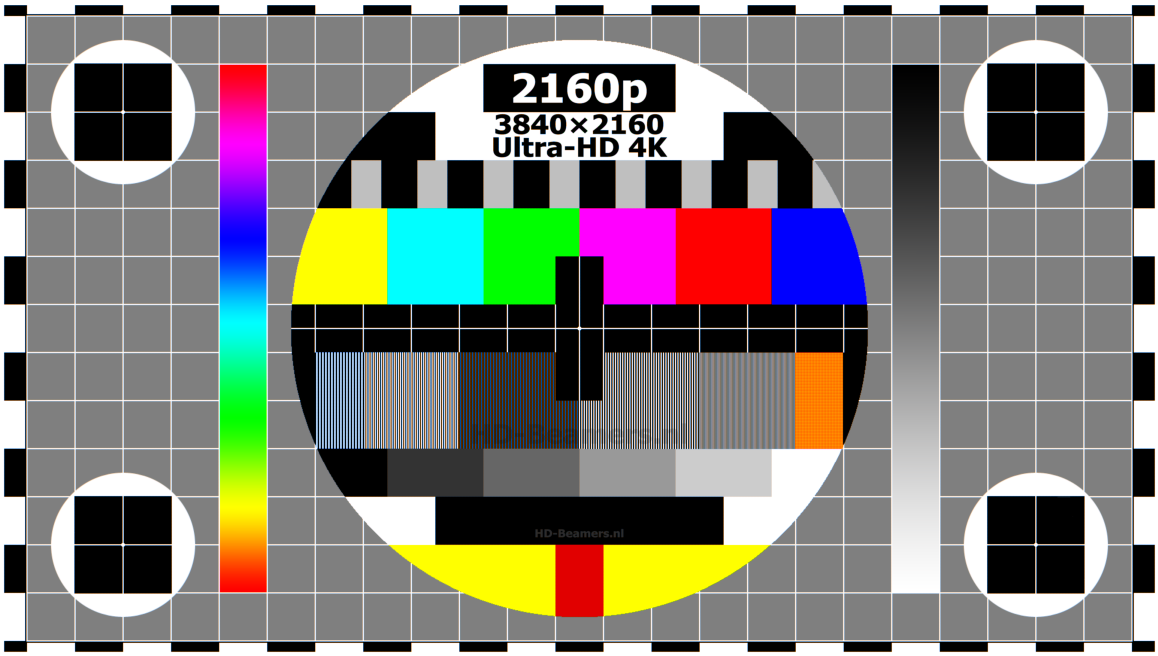
- $s = 1{,}0$ i $s = 1{,}5$: tylko $t = 0$, czyli brak interpolacji,
- $s = 2{,}0$: $t \in \{-1, 0\}$, czyli niesymetryczne okno 2×2,
- $s = 2{,}5$ i $s = 3{,}0$: $t \in \{-1, 0, 1\}$, czyli okno 3×3 (oba przypadki dają identyczny obraz).

Funkcje Namiot i Keys'a mają zera w ± 1 (a Keys'a także w ± 2), więc przy $s = 1$ również one dają niezerową wagę tylko samemu pikselowi. Wynik dla $s = 1{,}0$ (oraz dla Okna przy $s = 1{,}5$) jest więc po prostu obrazem matrycy z rys. 2 i 3. Dla porządku umieszczono te przypadki w tabelach, ale nie są one prawdziwym demozaikowaniem.

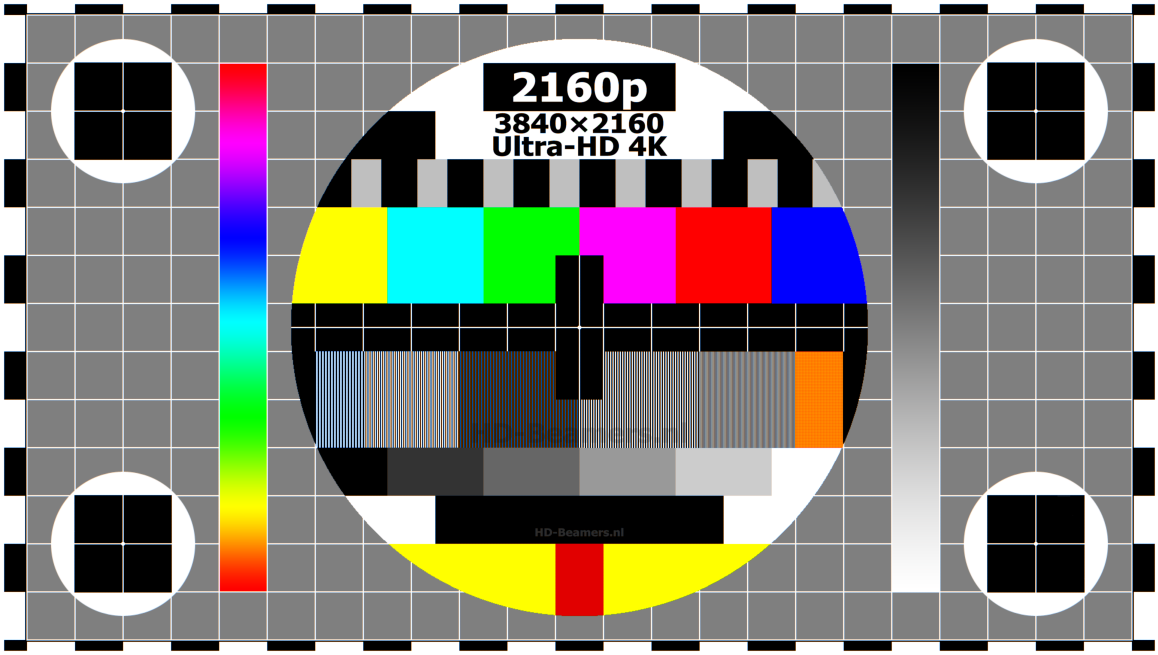
Żeby w żadnym pikselu nie zabrakło próbek, okno funkcji musi objąć najbliższe próbki każdego koloru. W filtrze Bayera wystarczy otoczenie 3×3. W filtrze X-Trans także wystarcza okno 3×3, dzięki opisanej wyżej własności tego wzoru. Natomiast niesymetryczne okno 2×2 (Okno, $s = 2$) zawodzi dla X-Trans: w $4/36 \approx 11{,}1\%$ położeń nie ma w nim żadnej próbki R ani B.

4 Wyniki demozaikowania

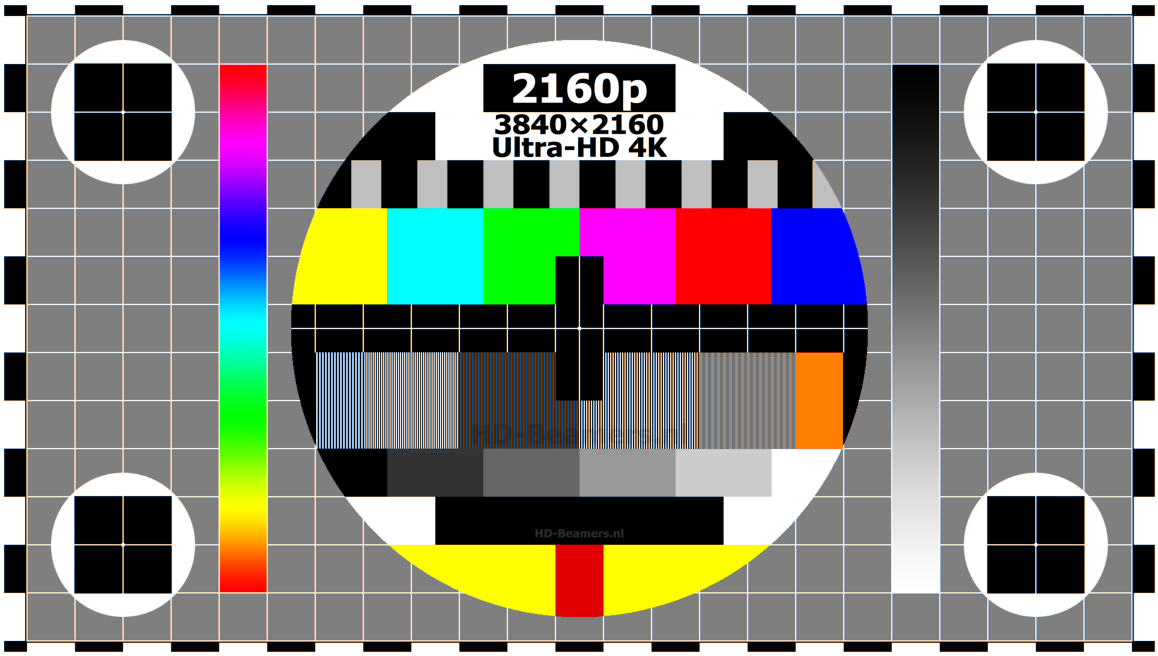
Wykonano demozaikowanie obu obrazów RAW wszystkimi trzema funkcjami dla skal $s \in \{1{,}0;\ 1{,}5;\ 2{,}0;\ 2{,}5;\ 3{,}0\}$, a dla funkcji Keys'a dodatkowo $s = 2{,}25$. Poniżej pokazano wybrane obrazy w całości (pełne obrazy są załączone do pliku PDF i można je otworzyć klikając na rysunek), a następnie powiększone fragmenty umożliwiające porównanie szczegółów.



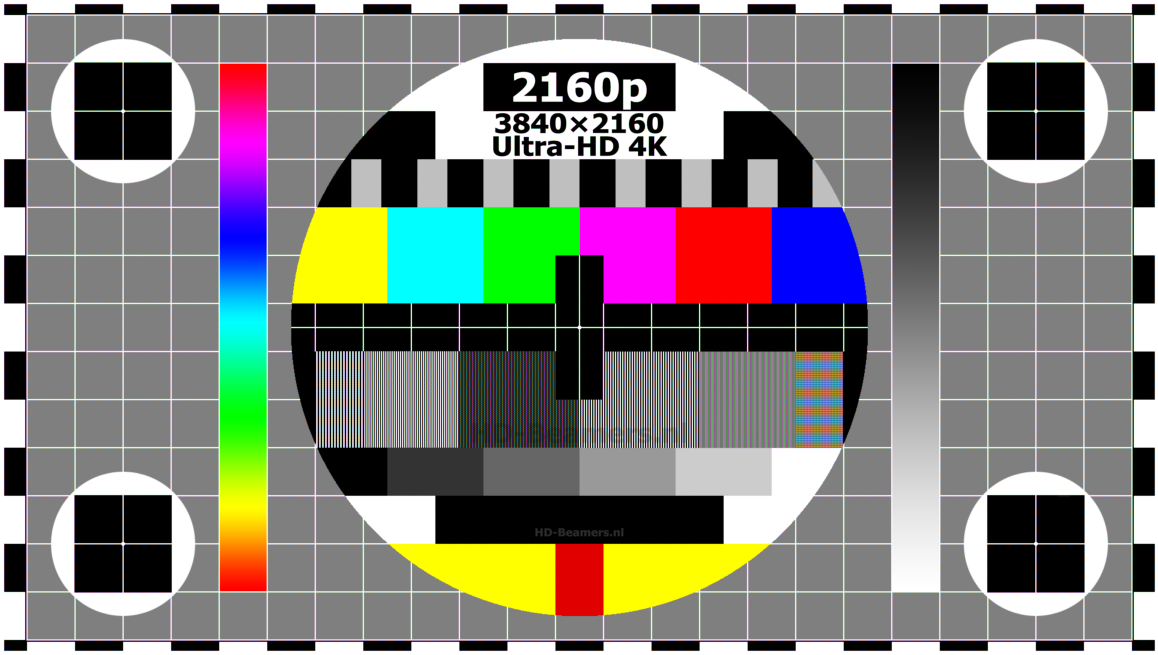
Rysunek 4: Bayer. Demozaikowanie algorytmem Okno, $s = 2{,}5$



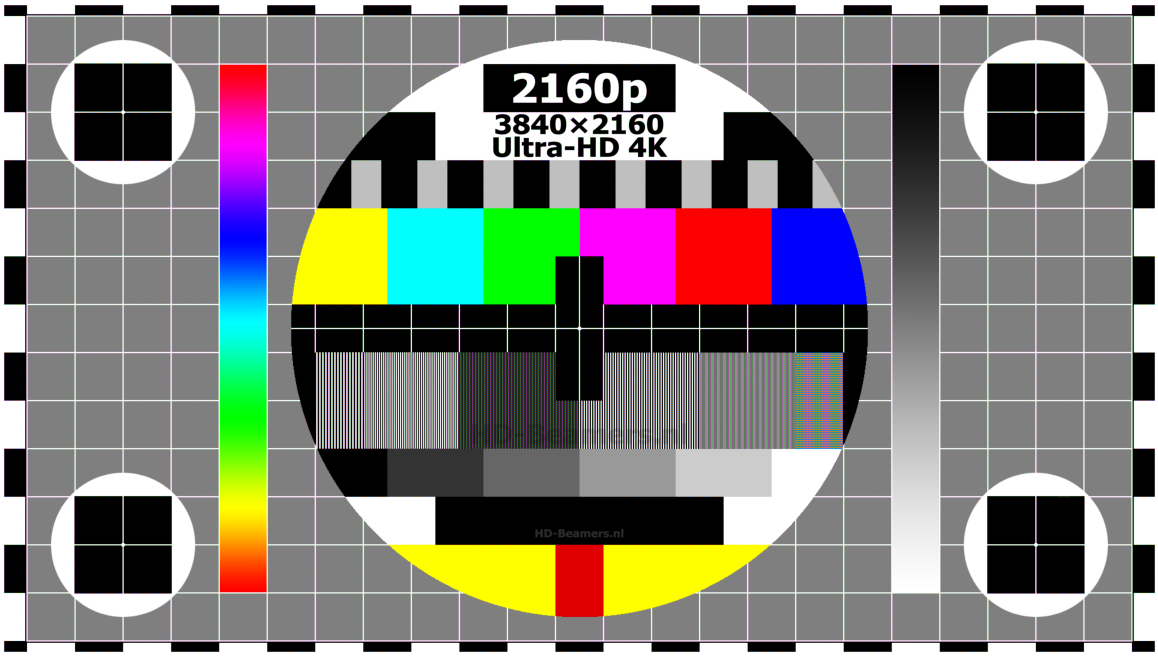
Rysunek 5: Bayer. Demozaikowanie algorytmem Namiot, $s = 1{,}5$



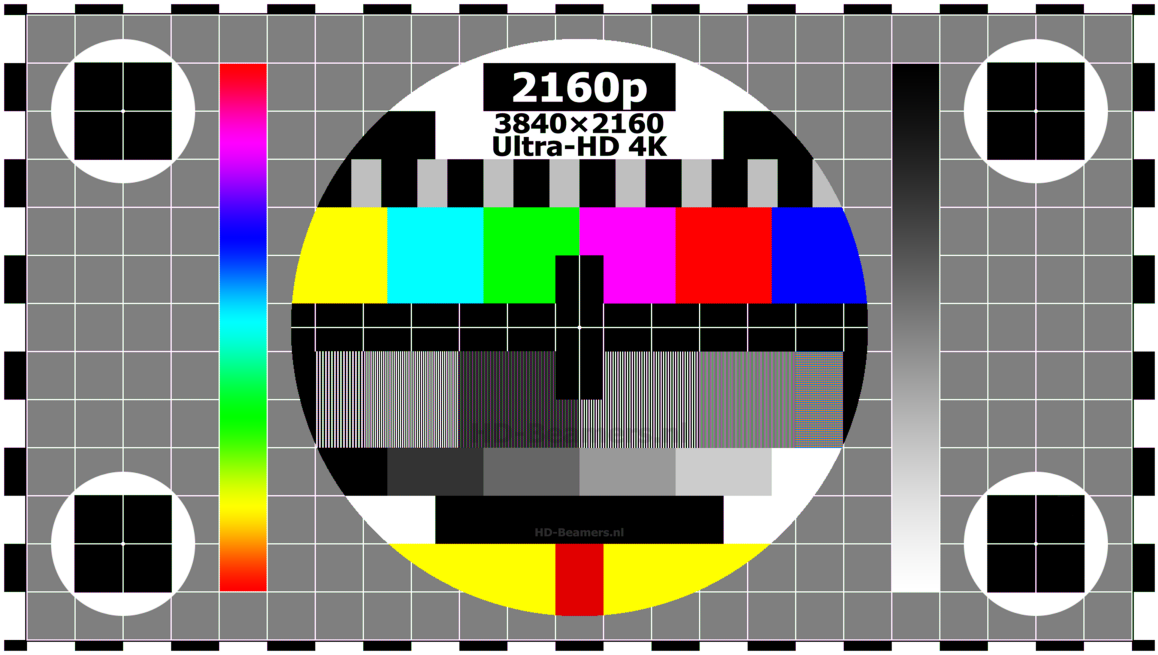
Rysunek 6: Bayer. Demozaikowanie algorytmem Keys'a, $s = 2{,}0$



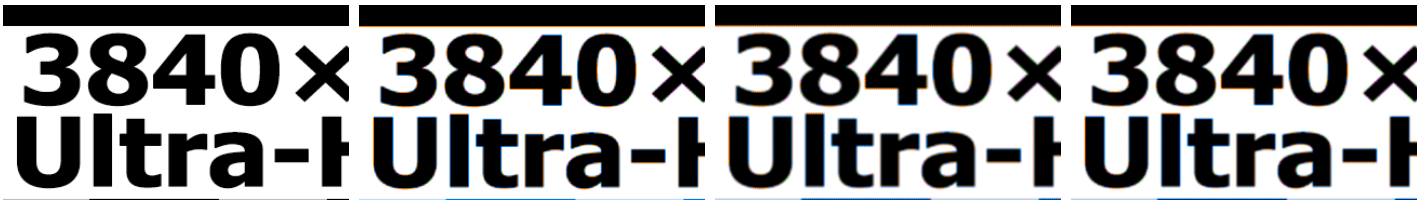
Rysunek 7: X-Trans. Demozaikowanie algorytmem Okno, $s = 2{,}5$



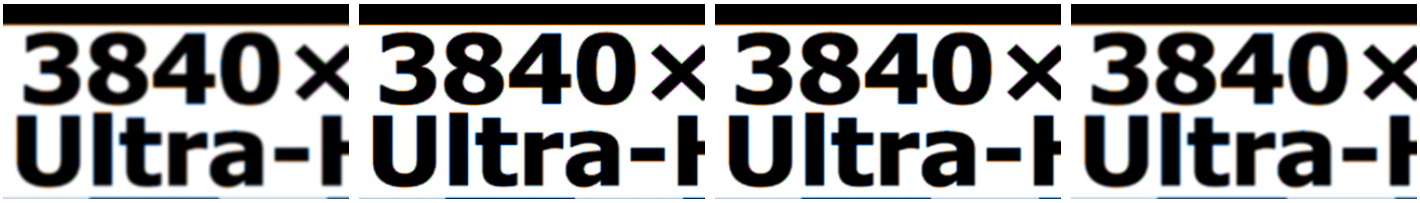
Rysunek 8: X-Trans. Demozaikowanie algorytmem Namiot, $s = 1{,}5$



Rysunek 9: X-Trans. Demozaikowanie algorytmem Keys'a, $s = 2{,}5$

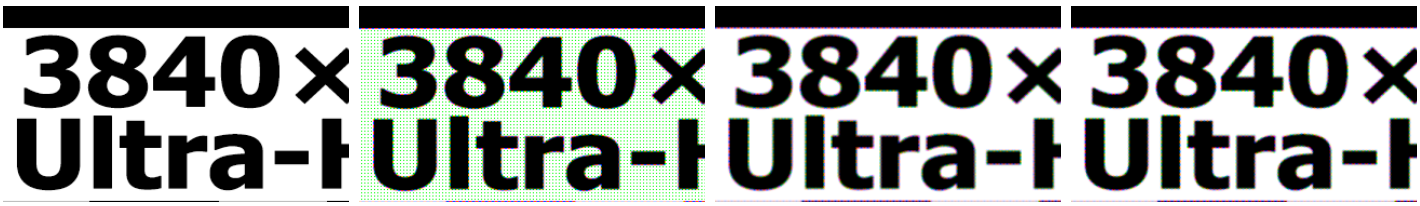


(a) Oryginał (b) Okno, $s = 2{,}0$ (c) Okno, $s = 2{,}5$ (d) Namiot, $s = 1{,}5$

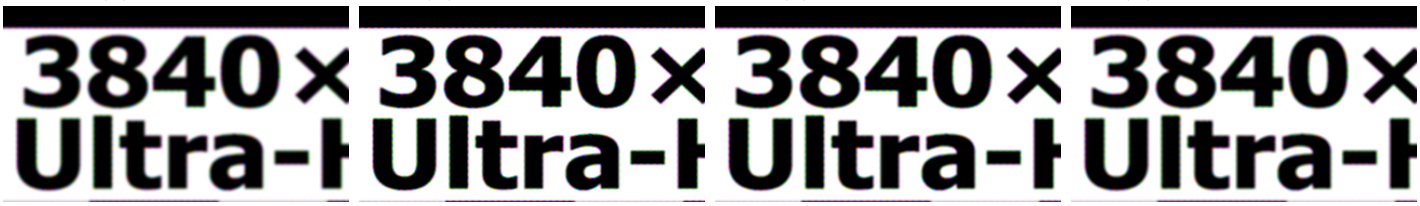


(e) Namiot, $s = 3{,}0$ (f) Keys, $s = 1{,}5$ (g) Keys, $s = 2{,}0$ (h) Keys, $s = 3{,}0$

Rysunek 10: Bayer. Powiększony fragment napisu (320 × 180 pikseli) po demozaikowaniu

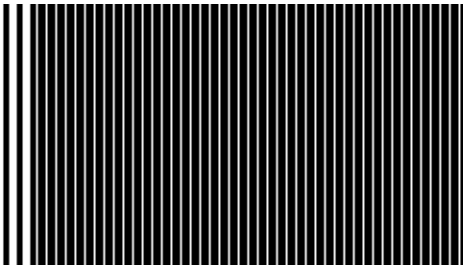


(a) Oryginał (b) Okno, $s = 2{,}0$ (c) Okno, $s = 2{,}5$ (d) Namiot, $s = 1{,}5$

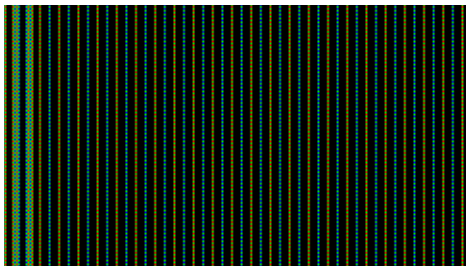


(e) Namiot, $s = 3{,}0$ (f) Keys, $s = 2{,}0$ (g) Keys, $s = 2{,}5$ (h) Keys, $s = 3{,}0$

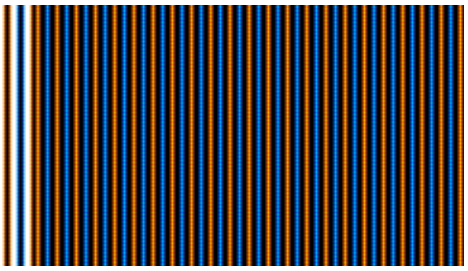
Rysunek 11: X-Trans. Powiększony fragment napisu (320 × 180 pikseli) po demozaikowaniu



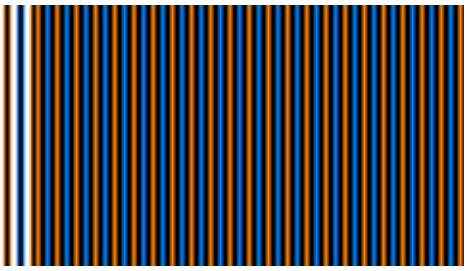
(a) Oryginał



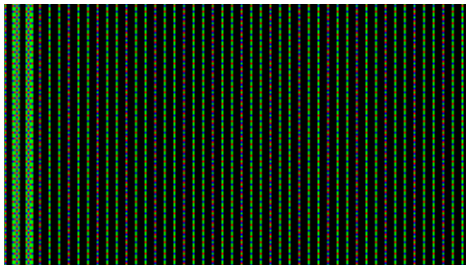
(b) Bayer, obraz matrycy



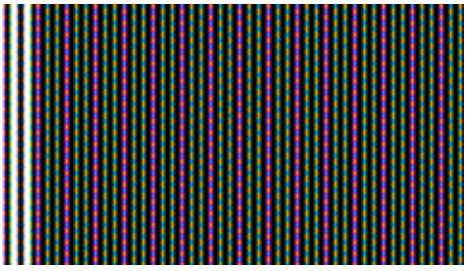
(c) Bayer, Namiot, $s = 1{,}5$



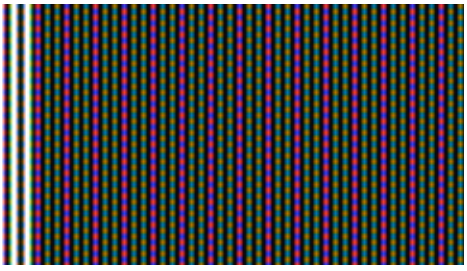
(d) Bayer, Keys, $s = 2{,}0$



(e) X-Trans, obraz matrycy



(f) X-Trans, Namiot, $s = 1{,}5$



(g) X-Trans, Keys, $s = 2{,}5$

Rysunek 12: Powiększony fragment klina rozdzielczości (240 × 135 pikseli)

5 Analiza wyników

5.1 Miary błędu

Ponieważ dostępny jest obraz oryginalny, każdy wynik można porównać z nim piksel po pikselu. Tak jak w poprzednim ćwiczeniu policzono średni błąd bezwzględny (MAE), błąd średniokwadratowy (RMSE) i szczytowy stosunek sygnału do szumu (PSNR), w skali jasności 0–255, uśrednione po wszystkich trzech kanałach. Odsetek zmienionych pikseli, użyty poprzednio, nie ma tu sensu, bo demozaikowanie zmienia praktycznie każdy piksel. Zamiast niego dodano miarę błędu barwy, która wychwytuje typowy defekt demozaikowania, czyli fałszywe kolory. Jest to RMSE różnic chrominancji ($R-G$, $B-G$) obrazu wynikowego względem oryginału. Szare piksele mają obie różnice równe zeru, więc każde zabarwienie szarego szczegółu zwiększa tę miarę, a sama zmiana jasności (np. rozmycie) wpływa na nią w niewielkim stopniu.

Algorytm	s	**MAE**	**RMSE**	**PSNR [dB]**	**Błąd barwy**
wszystkie (brak interpolacji)	1,0	81,26	125,59	6,15	137,90
Okno	2,0	5,19	31,69	18,11	25,34
	2,5	5,59	26,60	19,63	21,06
	3,0	5,59	26,60	19,63	21,06
Namiot	1,5	4,76	23,97	20,54	20,07
	2,0	5,09	24,75	20,26	19,40
	2,5	7,24	27,06	19,48	16,19
	3,0	8,16	28,80	18,94	15,66
Keys	1,5	**4,71**	**23,66**	**20,65**	20,56
	2,0	5,20	24,83	20,23	19,47
	2,25	8,24	29,25	18,81	**15,62**
	2,5	6,54	26,42	19,69	17,06
	3,0	7,73	28,42	19,06	15,93

Tabela 1: Bayer. Porównanie obrazów zdemozaikowanych z obrazem oryginalnym

Algorytm	s	**MAE**	**RMSE**	**PSNR [dB]**	**Błąd barwy**
wszystkie (brak interpolacji)	1,0	81,22	125,55	6,15	139,69
Okno	2,0	13,81	52,00	13,81	59,19
	2,5	6,15	27,46	19,36	21,33
	3,0	6,15	27,46	19,36	21,33
Namiot	1,5	**4,77**	**23,77**	**20,61**	23,19
	2,0	5,25	24,58	20,32	21,55
	2,5	7,19	25,27	20,08	14,49
	3,0	8,04	26,72	19,59	13,06
Keys	1,5	5,14	26,09	19,80	27,94
	2,0	5,43	24,90	20,21	21,78
	2,25	8,18	27,55	19,33	**12,72**
	2,5	6,49	25,37	20,04	17,14
	3,0	7,61	26,83	19,56	14,04

Tabela 2: X-Trans. Porównanie obrazów zdemozaikowanych z obrazem oryginalnym

Algorytm Okno dla $s = 1{,}5$ daje ten sam wynik co dla $s = 1{,}0$ (brak interpolacji), dlatego nie umieszczono go w tabelach osobno.

PSNR na poziomie ok. 20 dB może się wydawać niski, ale wynika głównie z klina rozdzielczości. Dla najlepszych wyników ok. 77% całkowitego błędu kwadratowego pochodzi z prostokąta zawierającego klin, który zajmuje tylko ok. 7% powierzchni obrazu. Poza nim PSNR wynosi ok. 26,5 dB (np. 26,64 dB dla Bayera i Namiotu przy $s = 1{,}5$, 26,48 dB dla X-Trans i Namiotu przy $s = 1{,}5$). Prążki o szerokości 1 piksela przekraczają częstotliwość Nyquista siatki próbek R i B (okres 2 piksele dla Bayera, a dla X-Trans średnio 3 piksele). Żaden algorytm interpolacji nie jest w stanie ich odtworzyć.

5.2 Bayer

Algorytm Okno nie tworzy nowych wartości, tylko powiela najbliższą próbkę. Dla $s = 2{,}0$ okno 2×2 jest niesymetryczne ($t \in \{-1{,}0\}$), więc każdy brakujący piksel R lub B dostaje wartość próbki leżącej na lewo lub powyżej. Każda próbka zostaje skopiowana na blok 2×2, ale bloki R i B są względem siebie przesunięte o jeden piksel w obu kierunkach. Na krawędziach kanały R i B zmieniają więc wartość w różnych miejscach. Widać to jako pomarańczowe i niebieskie obwódki wzdłuż krawędzi napisów (rys. 10b). Ma to też odbicie w mierze: kanał B ma tu PSNR tylko 15,4 dB, a kanał R 20,7 dB. Przy $s = 2{,}5$ okno jest już symetryczne (3×3), obwódki są słabsze, ale krawędzie pozostają kanciaste (rys. 10c).

Namiot i Keys'a dają podobne wyniki. Najlepsze miary uzyskano dla $s = 1{,}5$, przy czym Keys jest minimalnie lepszy (20,65 dB wobec 20,54 dB). Ciekawe jest porównanie $s = 1{,}5$ z $s = 2{,}0$ dla Namiotu. Dla kanałów R i B oba ustawienia dają *identyczny* wynik (PSNR 19,48 dB): próbki R/B leżą co 2 piksele, więc w obu przypadkach wagę mają tylko najbliższe próbki w odległości 1 (lub $\sqrt{2}$). Dzielone przez sumę wag dają tę samą średnią, czyli klasyczne demozaikowanie biliniowe. Różni się tylko kanał G: przy $s = 1{,}5$ znane próbki zielone dostają mniejszy udział sąsiadów (1/9 zamiast 1/4), więc G jest mniej rozmyty (24,07 dB wobec 22,45 dB). Dla $s = 2{,}0$ funkcje Namiot i Keys'a mają zera dokładnie w węzłach siatki R/B, więc oba algorytmy przechodzą przez znane próbki (interpolują dokładnie).

Zwiększanie s powyżej 2 rozmywa obraz (rys. 10e), więc PSNR spada. Jednocześnie maleje błąd barwy (z ok. 20 do ok. 16), bo przy szerszym jądrze wszystkie trzy kanały są rozmyte podobnie i różnice między nimi, czyli fałszywe kolory, się wygładzają. Jest to wymiana ostrości na wierność barw. Wynik dla funkcji Keys'a nie zmienia się z s monotonicznie: $s = 2{,}25$ daje najgorszy PSNR (18,81 dB), choć $s = 2{,}5$ jest lepsze (19,69 dB). Węzły $K(t/s)$ wypadają wtedy w niekorzystnych miejscach funkcji względem siatki próbek. Pokazuje to, że dla funkcji z ujemnymi płatami skalę trzeba dobierać ostrożnie.

5.3 X-Trans

Dla X-Trans algorytm Okno z $s = 2{,}0$ daje wyraźnie najgorszy wynik (13,81 dB). Zgodnie z wcześniejszą analizą w ok. 11% pikseli okno 2×2 nie zawiera żadnej próbki R ani B, więc obie składowe przyjmują wartość 0. W białych polach powstaje przez to regularna siatka zielonych kropek (rys. 11b). Dla $s \geq 2{,}5$ okno 3×3 zawsze zawiera próbki wszystkich kolorów i ten defekt znika.

Najlepszy PSNR (20,61 dB) daje Namiot z $s = 1{,}5$, czyli najmniejszą skalą, przy której każdy piksel ma w otoczeniu próbki wszystkich kolorów. Funkcja Keys'a wypada dla X-Trans gorzej niż Namiot przy każdej skali. Dla $s = 1{,}5$ ma ona zarówno niższy PSNR (19,80 dB), jak i największy błąd barwy (27,94). Ujemne płaty funkcji Keys'a są zaprojektowane dla regularnej siatki próbek. Przy nieregularnym rozkładzie próbek X-Trans ujemne wagi nie znoszą się symetrycznie, mianownik bywa mały i błędy zostają wzmocnione. Najlepszy wynik Keys osiąga dopiero przy $s = 2{,}0$ (20,21 dB).

Tak jak dla Bayera, większe s zmniejsza błąd barwy (do 13,06 dla Namiotu przy $s = 3{,}0$) kosztem ostrości.

5.4 Porównanie filtrów Bayera i X-Trans

Najlepsze wyniki dla obu filtrów są niemal identyczne pod względem PSNR (20,65 dB dla Bayera i 20,61 dB dla X-Trans). Błąd barwy liczony piksel po pikselu jest nawet nieco większy dla X-Trans przy małych skalach. Na rys. 12 widać jednak wyraźną różnicę w *charakterze* fałszywych kolorów w klinie rozdzielczości. Dla Bayera białe prążki zamieniają się w regularne, naprzemiennie pomarańczowe i niebieskie pasy, czyli

barwną morę rozciągniętą na cały obszar klina. Dla X-Trans fałszywe kolory tworzą drobną, nieregularną mozaikę punktów zielonych, purpurowych, czerwonych i niebieskich, która z daleka zlewa się w kolor zbliżony do szarego.

Żeby to zmierzyć, błąd chrominancji uśredniono najpierw w oknie 12×12 pikseli. Usuwa to drobny „szum" kolorów i zostawia tylko składowe barwne o niskiej częstotliwości, czyli morę widoczną z normalnej odległości. Wyniki zestawiono w tabeli 3.

Algorytm	s	**Cały obraz**		**Klin rozdzielczości**	
		Bayer	**X-Trans**	**Bayer**	**X-Trans**
Okno	2,5	10,57	3,81	39,06	9,75
Namiot	1,5	10,56	3,51	39,05	8,98
Namiot	2,0	10,56	3,67	39,04	9,39
Namiot	3,0	10,47	3,66	38,79	9,16
Keys	2,0	10,56	3,86	39,05	9,82

Tabela 3: Niskoczęstotliwościowy błąd barwy (barwna mora): RMSE różnic chrominancji uśrednionych w oknie 12×12 pikseli

Barwna mora jest dla X-Trans ok. 3 razy mniejsza w całym obrazie i ok. 4 razy mniejsza w klinie rozdzielczości, niezależnie od wybranej funkcji i skali. Jest to zgodne z deklarowaną zaletą tego filtru. Regularny wzór Bayera próbkuje R i B z okresem 2 piksele w każdym kierunku, więc szczegóły o okresie zbliżonym do 2 pikseli zamieniają się w jednolity, fałszywy kolor. Nieregularny wzór X-Trans rozprasza ten błąd na drobny wzór o wysokiej częstotliwości, który oko odbiera jako szum, a nie jako barwną plamę. Dlatego aparaty z matrycą X-Trans mogą obywać się bez optycznego filtra antyaliasingowego. Na rys. 10 i 11 widać też, że obwódki na krawędziach napisów są dla X-Trans słabsze i mniej nasycone (purpurowo-zielone zamiast pomarańczowo-niebieskich).

6 Wnioski

Wybór algorytmu. Za najlepszy algorytm uznano **znormalizowany splot z funkcją Namiot przy skali** $s = 1{,}5$, dla obu filtrów CFA. Uzasadnienie:

- Dla X-Trans daje najlepsze wartości wszystkich trzech miar MAE, RMSE i PSNR (tab. 2). Dla Bayera ustępuje funkcji Keys'a o zaledwie 0,11 dB PSNR (tab. 1). Różnica ta jest wizualnie niezauważalna.
- Przy $s = 1{,}5$ funkcja obejmuje dokładnie otoczenie 3×3, czyli najmniejsze, w którym zawsze są próbki wszystkich kolorów. Zapewnia to brak „dziur" (zerowych mianowników) przy najmniejszym możliwym rozmyciu.
- Wszystkie wagi Namiotu są nieujemne, więc wynik jest zawsze średnią ważoną sąsiednich próbek. Nie powstają przeregulowania i algorytm jest stabilny także dla nieregularnej siatki X-Trans, na której funkcja Keys'a wypada gorzej.
- Jest najprostszy obliczeniowo z algorytmów dających dobre wyniki (okno 3×3 zamiast 5×5 lub większego dla Keys'a).

Jest to inny wniosek niż w poprzednim ćwiczeniu, w którym we wszystkich operacjach najlepsza była funkcja Keys'a. Przy skalowaniu i obrotach próbki tworzą pełną, regularną siatkę, a ujemne płaty funkcji Keys'a podbijają krawędzie i dają ostrzejszy obraz. Przy demozaikowaniu próbki danego koloru są rzadkie i (dla X-Trans) nieregularne. Funkcja Keys'a traci wtedy swoją podstawową własność dokładnej interpolacji, a normalizacja przez sumę wag z ujemnymi składnikami pogarsza jej stabilność.

Algorytm Okno okazał się najgorszy. Nawet przy optymalnej skali ($s \geq 2{,}5$) daje kanciaste krawędzie i fałszywe kolory, a przy $s = 2{,}0$ niesymetryczne okno sprawia, że kanały R i B zmieniają się na krawędziach w różnych miejscach (Bayer) lub zostawia piksele bez próbek R i B (X-Trans).

Jeśli ważniejszy od ostrości jest brak fałszywych kolorów, można zwiększyć skalę do $s = 2{,}5$–$3{,}0$. Błąd barwy spada wtedy o ok. 20–45% kosztem ok. 1–1,6 dB PSNR i widocznego rozmycia.

Bayer czy X-Trans. Pod względem dokładności piksel po pikselu oba filtry dają po demozaikowaniu prawie ten sam wynik. X-Trans daje jednak wyraźnie mniejszą barwną morę (ok. 3–4 razy, tab. 3). Ma to znaczenie przy fotografowaniu drobnych, regularnych wzorów, np. tkanin czy siatek. Ceną jest bardziej złożony i kosztowniejszy algorytm demozaikowania. Prosta interpolacja dwuliniowa Bayera ma zamkniętą postać, a dla X-Trans potrzebne jest ogólne podejście z maską, takie jak znormalizowany splot.

Możliwe ulepszenia. Opisane metody interpolują każdy kanał niezależnie. Pomijają w ten sposób silną korelację między kanałami w rzeczywistych obrazach: krawędzie występują zwykle w tych samych miejscach we wszystkich trzech kanałach. Lepsze wyniki dają m.in.:

- interpolacja różnic kolorów ($R - G$, $B - G$) zamiast samych kanałów R i B. Różnice te zmieniają się w obrazie znacznie wolniej niż same kanały, więc ich interpolacja daje mniej fałszywych kolorów,
- metody kierunkowe (np. Hamiltona–Adamsa), które interpolują wzdłuż krawędzi, a nie w poprzek nich,
- metoda Malvara–He–Cutlera, będąca interpolacją dwuliniową z korekcją gradientem z pozostałych kanałów,
- dla X-Trans algorytm Markesteijna, stosowany m.in. w programach dcraw i RawTherapee.

Prostym ulepszeniem obecnego programu jest też pozostawienie znanych próbek bez zmian (dla $M_c(x,y) = 1$ przepisywać wartość z obrazu RAW), co usuwa niepotrzebne rozmycie kanału G.